

VeloGrad: Local Value-Gradient Learning for Reinforcement Fine-Tuning of Flow Policies

Anonymous Authors

Abstract—Value-based reinforcement fine-tuning of flow policies must connect final-action values to the local velocities that generate them. We introduce VeloGrad, which learns this connection with a velocity-conditioned critic. The critic predicts the action value obtained by applying a candidate velocity for one flow step and completing generation under a frozen copy of the current policy. Since fitting values alone does not directly supervise the derivative used by the actor, we also train directional derivatives against normalized action-value differences from symmetric finite-radius velocity perturbations and their downstream continuations. Actor updates use the learned derivative without backpropagating through the remaining flow solver; deployment uses the standard sampler without critic queries. Experiments cover offline and offline-to-online learning on OGBench and Robomimic. VeloGrad achieves 79.6% average offline success and 97.7% after online adaptation across four OGBench domains. On a Piper-X robot, it achieves 29/30 success on disturbed flower insertion and 21/30 autonomous end-to-end success on Windmill Activation, compared with 16/30 for DSRL.

Index Terms—Flow matching, reinforcement learning, robot manipulation, policy fine-tuning, value gradient

I. INTRODUCTION

Diffusion and flow policies learn complex, multimodal robot behaviors from demonstrations [1], [2]. Yet these policies can fail when targets move during execution, dynamic interactions require precise control, or errors accumulate across manipulation stages. Reward feedback from these executions offers a way to improve behavior without collecting new demonstrations for every failure. We study reinforcement learning (RL) for fine-tuning pretrained flow policies in this setting.

The optimization challenge is to connect the quantity an action critic evaluates to the quantity a flow policy predicts. The critic scores final actions, but the policy generates them through a sequence of velocity predictions. A local velocity change affects the final action through all remaining flow steps. Backpropagating through those steps can be costly and sensitive to the conditioning of the flow dynamics [3], [4]. Existing methods use auxiliary actors or latent steering [3], [5], redesign differentiable samplers [6], or train flow fields with local objectives [4], [7], [8].

VeloGrad learns the downstream effect of a local velocity change. A velocity critic predicts the action-critic value obtained by applying a candidate velocity for one flow step and completing generation under a frozen copy of the current policy. Fitting scalar values alone leaves the derivative used by the actor without direct supervision. We therefore perturb the current velocity in two opposite directions, complete both continuations, and supervise the critic’s directional derivative

with their normalized action-value difference. The actor updates through this learned derivative without backpropagating through the remaining solver.

Our contributions are:

- 1) **Velocity-conditioned continuation values.** We define and learn the downstream action value of a candidate flow velocity, giving the actor a local value gradient that accounts for the remaining generation dynamics.
- 2) **Finite-radius derivative supervision.** We complement center-value regression with directional sensitivity targets from symmetric velocity perturbations and their downstream action values.
- 3) **Simulation and real-robot evaluation.** Across OGBench and Robomimic, we evaluate offline and offline-to-online learning. VeloGrad reaches 79.6% and 97.7% average OGBench success in the two settings, compared with 66.5% and 94.7% for QAM. Real-robot tests cover target disturbance, elastic puck launching, and Windmill Activation, with 21/30 autonomous Windmill successes versus 16/30 for DSRL, the strongest baseline.

II. RELATED WORK

Real-world reinforcement fine-tuning of robot policies.

Off-policy RL reuses experience and demonstrations to improve data efficiency [9]. Robotic learning systems combine this approach with visual perception and reward specification [10], using human corrections to support exploration in contact-rich and dynamic manipulation [11]. Recent methods adapt demonstration-trained generative policies [5], [12], [13] and vision-language-action models [14]–[16] through RL. Broader post-training pipelines use iterative data collection and policy refinement [17] or specialist-to-generalist distillation [18]. VeloGrad focuses on the policy update within this setting: turning terminal action values into local velocity gradients.

Reinforcement learning with diffusion and flow policies.

Offline methods combine generative behavior modeling with value optimization or behavior constraints [19], [20]; policy-gradient methods optimize denoising or flow-matching objectives [21]–[23]. Other approaches separate behavior modeling from value maximization through one-step actors [3], [24], learn values over action chunks [25], or adapt policies through residual, edit-based, or latent interfaces [5], [12], [26]–[28].

Value-gradient methods [29] extend critic-based policy improvement to score or velocity supervision [4], [7], [8], [30] and differentiable generation or latent mappings [6], [31]. Prior work also studies the gap between proxy-value accuracy

and useful policy gradients [31], with derivative supervision developed more broadly in Sobolev training [32]. VeloGrad combines an explicit scalar continuation value conditioned on a candidate velocity with directional derivative targets from finite-radius downstream evaluations.

III. METHOD

A. Reinforcement Learning with Flow Policies

We consider a flow policy $v_\theta(s, x_t, t)$ over a d -dimensional action representation. Here, velocity is the rate of change of the action representation along flow time. Given robot state s and noise $x_0 \sim \mathcal{N}(0, I)$, we generate actions with K Euler steps, $\Delta t = 1/K$, and $t_k = k/K$:

$$x_{k+1} = x_k + \Delta t v_\theta(s, x_k, t_k), \quad (1)$$

for $k = 0, \dots, K-1$. Each Euler update uses elementwise clipping, omitted from (1) for readability. The terminal output $a = x_K \in \mathbb{R}^d$ is clipped to the action bounds and executed. In all experiments, a is an H -step action chunk with per-step dimension d_a , so $d = Hd_a$; $H = 1$ recovers single-step actions.

For flow-matching pretraining, we sample a demonstration action representation a_D , a flow time $t \sim \text{Unif}(0, 1)$, and noise $x_0 \sim \mathcal{N}(0, I)$. We construct the interpolated training state

$$x_t^D = (1-t)x_0 + ta_D. \quad (2)$$

The flow-matching objective $\mathcal{L}_{\text{FM}}$ regresses $v_\theta(s, x_t^D, t)$ toward the target velocity $a_D - x_0$ using mean-squared error. During RL, a_D is a recorded action chunk sampled from the replay buffer, which contains offline data and is augmented with new experience during online adaptation.

Following Q-Chunking [25], the action critic evaluates the entire H -step action chunk. For a replay segment with initial state s , action chunk a , rewards $r_0, \dots, r_{H-1}$, and next state s' after the chunk, we sample $a' \sim \pi_\theta(\cdot | s')$ using the current flow policy and minimize the head-wise TD loss

$$y_{\text{TD}} = \sum_{j=0}^{H-1} \gamma^j r_j + \gamma^H m \bar{Q}_{\phi^-}^A(s', a'), \quad (3)$$

$$\mathcal{L}_{Q^A} = \mathbb{E} \left[\left(Q_\phi^A(s, a) - \text{sg}(y_{\text{TD}}) \right)^2 \right],$$

where m is the bootstrap mask, ϕ^- denotes target-critic parameters, and $\bar{Q}_{\phi^-}^A$ is the mean of the two target heads. Terminal transitions suppress bootstrapping; rewards use the corresponding termination masks. Head indices are omitted.

VeloGrad learns a velocity critic to translate terminal action value into local flow updates (Fig. 1).

B. VeloGrad: Local Value-Gradient Learning

Local velocity value. For an intermediate flow state x , flow time t , and candidate velocity u , we define the local continuation value as

$$Q_{\bar{\theta}}(s, x, t, u) = Q_\phi^A \left(s, \Phi_{\bar{\theta}}^{t+h(t) \rightarrow 1}(x + h(t)u) \right), \quad (4)$$

$$h(t) = \min \left(\frac{1}{K}, 1-t \right),$$

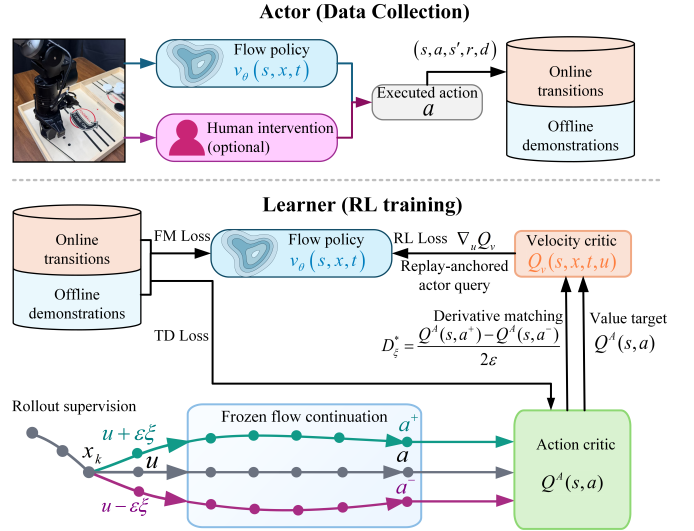


Fig. 1. Overview of VeloGrad. Top: the actor collects transitions with optional human intervention. Bottom: the learner trains a velocity critic on frozen flow continuations and updates the flow policy with the critic derivative $\nabla_u Q_v$.

where $\bar{\theta} = \text{sg}(\theta)$, and $\Phi_{\bar{\theta}}^{\tau \rightarrow 1}$ denotes the downstream continuation from time τ to the terminal action under the frozen current flow policy. It clips its input (the state after the candidate step), applies the remaining Euler steps with per-step clipping, and clips the terminal action, including when $\tau = 1$. Its Jacobian includes these clipping operations.

Let $y = x + h(t)u$ and $a = \Phi_{\bar{\theta}}^{t+h(t) \rightarrow 1}(y)$. At points where the composed downstream mapping is differentiable,

$$\nabla_u Q_{\bar{\theta}} = h(t) \left(\frac{\partial a}{\partial y} \right)^\top \nabla_a Q_\phi^A(s, a). \quad (5)$$

Equation (5) expresses the local value gradient through the terminal action-value gradient and the remaining flow dynamics. We learn a velocity critic $Q_{v,\psi}(s, x, t, u)$ to represent the corresponding continuation value directly in velocity space.

Velocity critic learning. To train $Q_{v,\psi}$, we draw Gaussian rollout noise $x_0^\pi \sim \mathcal{N}(0, I)$ and a solver step k for each sample, roll out the frozen current policy to (x_k, t_k) , and set $u = v_{\bar{\theta}}(s, x_k, t_k)$. Continuing the rollout to the terminal flow output $a_K = x_K$, we anchor the local velocity value with

$$\mathcal{L}_{\text{center}} = \left[Q_{v,\psi}(s, x_k, t_k, u) - \text{sg}(Q_\phi^A(s, a_K)) \right]^2. \quad (6)$$

Value matching alone does not directly supervise the velocity gradient used by the actor. We add finite-radius perturbations in the d -dimensional velocity space:

$$\xi \sim \text{Unif}(\mathbb{S}^{d-1}), \quad u^\pm = u \pm \epsilon \xi.$$

We apply each perturbed velocity for one step and continue the frozen flow to terminal actions a^+ and a^- . Their action values define the directional downstream-value difference

$$D_\xi^* = \frac{Q_\phi^A(s, a^+) - Q_\phi^A(s, a^-)}{2\epsilon}. \quad (7)$$

We treat D_ξ^* as a stop-gradient target for the velocity critic’s directional derivative,

$$\mathcal{L}_{\text{sens}} = \left[\xi^\top \nabla_u Q_{v,\psi}(s, x_k, t_k, u) - D_\xi^* \right]^2. \quad (8)$$

The resulting velocity-critic objective is

$$\mathcal{L}_{Q_v} = \mathcal{L}_{\text{center}} + \beta \mathcal{L}_{\text{sens}}. \quad (9)$$

For fixed (s, x_k, t_k, u) , write $f(w) = \mathcal{Q}_{\bar{\theta}}(s, x_k, t_k, w)$ and $g = \nabla_u Q_{v,\psi}(s, x_k, t_k, u)$. Define its ball-smoothed value as $f_\epsilon(u) = \mathbb{E}_{b \sim \text{Unif}(\mathbb{B}^d)}[f(u + \epsilon b)]$, where $\mathbb{B}^d$ is the unit ball. For locally Lipschitz f ,

$$\begin{aligned} \nabla_u f_\epsilon(u) &= d \mathbb{E}_\xi[\xi D_\xi^*], \\ \mathbb{E}_\xi[\mathcal{L}_{\text{sens}}] &= \frac{1}{d} \|g - \nabla_u f_\epsilon(u)\|_2^2 + C, \end{aligned} \quad (10)$$

where C is independent of ψ . The first identity follows from ball smoothing [33] and spherical symmetry; expanding the square with $\mathbb{E}_\xi[\xi \xi^\top] = I/d$ gives the second. Thus, the finite-radius loss supervises the smoothed continuation-value gradient. The perturbed velocities are used only to construct directional targets, without additional value regression at $u^\pm$.

Local actor improvement. We train the velocity critic on current-policy rollout states at discrete solver times. Actor updates query it on replay-anchored continuous states from (2), with $t \sim \text{Unif}(0, 1)$. Section IV ablates this query-distribution design.

The actor objective is

$$\begin{aligned} \mathcal{L}_{\text{actor}} &= \lambda_{\text{FM}} \mathcal{L}_{\text{FM}} \\ &\quad - \lambda_Q \bar{Q}_{v,\psi}(s, x_t^D, t, v_\theta(s, x_t^D, t)), \end{aligned} \quad (11)$$

where $\bar{Q}_{v,\psi}$ denotes the aggregated velocity-critic value.

With the velocity critic fixed during the actor update, the unweighted value term yields

$$\nabla_\theta [-\bar{Q}_{v,\psi}] = - \left(\frac{\partial v_\theta}{\partial \theta} \right)^\top \nabla_u \bar{Q}_{v,\psi}. \quad (12)$$

The actor follows the learned velocity gradient without back-propagating through the downstream flow solver. Algorithm 1 summarizes training. Random draws are per sample; update frequencies are specified in the experimental setup.

Implementation. Both critics use two heads with head-wise regression and mean aggregation. Both critic losses are computed before the joint update, with Q_v targets formed from the pre-update action critic and treated as stop-gradient. The target action critic is initialized from Q_ϕ^A and updated by Polyak averaging at every critic update (Algorithm 1), with $\tau_Q = 0.005$ in all experiments. Each velocity-critic head receives the corresponding action-critic head’s value and sensitivity targets, while the actor uses their mean, $\bar{Q}_{v,\psi}$. We sample one spherical direction in the d -dimensional flow-output velocity space per local query. All VeloGrad experiments use $K = 10$ Euler steps, $\beta = 1$, and $\lambda_Q = 1$; the default perturbation radius is $\epsilon = 0.1$. On OGBench, $\lambda_{\text{FM}} = 10$ in both phases, except for Scene task 4 (25 in both phases) and Cube-Double/Triple during online adaptation (50). We use $\lambda_{\text{FM}} = 100$ on Robomimic Lift/Can/Square, 750 on Transport,

Algorithm 1 VeloGrad Training

Require: Replay buffer $\mathcal{D}$, flow actor v_θ , action critic Q_ϕ^A , velocity critic $Q_{v,\psi}$

- 1: Initialize target action-critic parameters $\phi^- \leftarrow \phi$
- 2: **for** each training iteration **do**
- 3: Sample a replay minibatch $(s, a_D, \{r_j\}, s', m)$ from $\mathcal{D}$
- 4: Sample $a' \sim \pi_\theta(\cdot | s')$ via (1)
- 5: Compute $\mathcal{L}_{Q^A}$ using (3) with $a = a_D$
- 6: Freeze the current flow actor $\bar{\theta} \leftarrow \text{sg}(\theta)$
- 7: Sample $x_0^\pi \sim \mathcal{N}(0, I)$ and $k \sim \text{Uniform}\{0, \dots, K-1\}$
- 8: Roll out $v_{\bar{\theta}}$ from x_0^π to obtain (x_k, t_k)
- 9: $u \leftarrow v_{\bar{\theta}}(s, x_k, t_k)$
- 10: Continue with u to $a_K = x_K$; compute $\mathcal{L}_{\text{center}}$ via (6)
- 11: Sample $\xi \sim \text{Uniform}(\mathbb{S}^{d-1})$; set $u^\pm \leftarrow u \pm \epsilon \xi$
- 12: Continue the frozen flow from (x_k, t_k) with local velocities $u^\pm$ to obtain $a^\pm$
- 13: Compute D_ξ^* , $\mathcal{L}_{\text{sens}}$, and $\mathcal{L}_{Q_v}$ via (7)–(9)
- 14: Jointly update the critics using $\mathcal{L}_{Q^A} + \mathcal{L}_{Q_v}$
- 15: Update the target: $\phi^- \leftarrow (1 - \tau_Q)\phi^- + \tau_Q \phi$
- 16: Draw fresh $t \sim \text{Unif}(0, 1)$ and $x_0 \sim \mathcal{N}(0, I)$
- 17: $x_t^D \leftarrow (1 - t)x_0 + ta_D$
- 18: Update θ using $\mathcal{L}_{\text{actor}}$ in (11)
- 19: **end for**

and 100 on all real-robot tasks. Downstream continuations use the generation solver and clipping rules defined above. Evaluation and deployment use the standard flow sampler without critic queries.

IV. EXPERIMENTS

A. Simulation Experiments

Setup. We first evaluate VeloGrad on four OGBench manipulation domains [34]: Scene, Puzzle-3 $\times$ 3, Cube-Double, and Cube-Triple. All methods use action chunks of $H = 5$ and the default `-play` datasets. The Cube domains use dense rewards, whereas Scene and Puzzle-3 $\times$ 3 use sparse rewards. Offline RL is trained for 1M gradient steps without environment interaction. For offline-to-online adaptation, each method starts from its own offline checkpoint and continues training with the same 1M-environment-step interaction budget under the same evaluation protocol. Method-specific update, replay, and optimization strategies are retained. We reproduce prior methods using the released QAM codebase and follow the experimental and evaluation protocol used in QAM [4].

On OGBench, DAC and DSRL use the configurations provided by the released QAM implementation, while QAM and QC-FQL follow their published configurations. For VeloGrad, we search λ_{FM} over $\{10, 25, 50, 75, 100\}$ on OGBench and $\{100, 250, 500, 750, 1000\}$ on Robomimic using four preliminary seeds. Configurations are selected based on success rates and inspection of critic estimates for signs of overestimation, then fixed before the reported multi-seed evaluations.

Unless otherwise specified, we report success rate at the final checkpoint. Table I reports mean $\pm$ standard deviation over 12 training seeds: for each seed, success rates are first

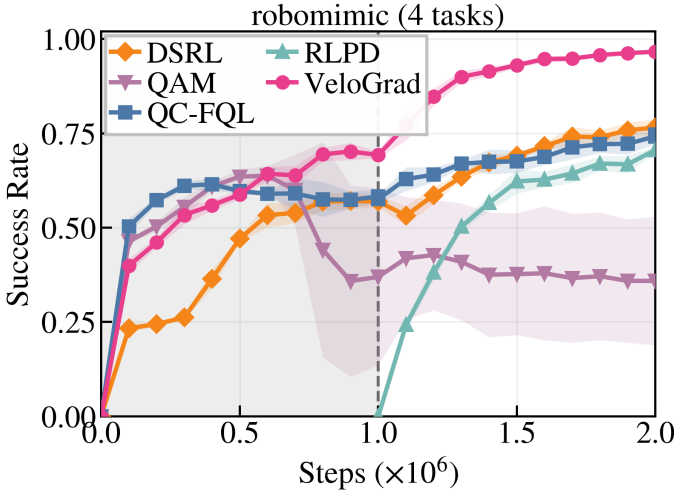


Fig. 2. Robomimic MH success with low-dimensional observations, averaged over Lift, Can, Square, and Transport within each seed, then reported as mean and sample standard deviation across four training seeds per method. Shading denotes offline training; the dashed line marks the start of online adaptation.

averaged over the five tasks within a domain, followed by aggregation across seeds. Design-choice experiments are conducted on offline Cube-Triple, while the directional-sensitivity ablation covers Cube-Double and Cube-Triple.

Main results. VeloGrad leads the compared methods in average OGBench success in both settings (Table I): $79.6 \pm 1.7\%$ offline, compared with $66.5 \pm 0.7\%$ for QAM, and $97.7 \pm 1.8\%$ after online adaptation. Its largest gain is on Cube-Triple, reaching $44.7 \pm 4.6\%$ offline versus $4.8 \pm 1.4\%$ for QAM, and 93.4% after adaptation versus $79.4 \pm 0.7\%$ for QAM. It remains competitive on the other domains, with near-ceiling performance on most after adaptation.

We further evaluate VeloGrad on Lift, Can, Square, and Transport using the Robomimic multi-human (MH) datasets with low-dimensional observations [35]. The action-chunk horizon is $H = 5$ for Lift, Can, and Square, and $H = 8$ for Transport. Per-task means and sample standard deviations are provided in Supplementary Tables S1 and S2. Training comprises 1M offline gradient steps followed by 1M environment steps of online adaptation. We compare against DSRL, QAM, QC-FQL, and RLPD, with RLPD evaluated only in O2O. QAM improves early offline but then degrades, remaining well below the other methods after the transition. VeloGrad leads the compared methods after online adaptation, reaching $96.7 \pm 0.5\%$ versus $76.6 \pm 2.2\%$ for DSRL. The largest separation is on Transport: 92.3% versus 25.2% for QC-FQL. VeloGrad also reaches 98.9% on Square and remains competitive on Lift and Can.

Design choices. VeloGrad queries the velocity critic at intermediate flow states during actor updates. We compare three query-state constructions. **Policy-Discrete** uses x_k^π generated by current-policy rollouts on the solver grid; **Data-Discrete** constructs x_k^D from replay action chunks at discrete solver times t_k ; and **Data-Continuous** samples the replay-anchored state x_t^D defined in (2) at $t \sim \mathcal{U}(0, 1)$. As shown in Fig. 3(a),

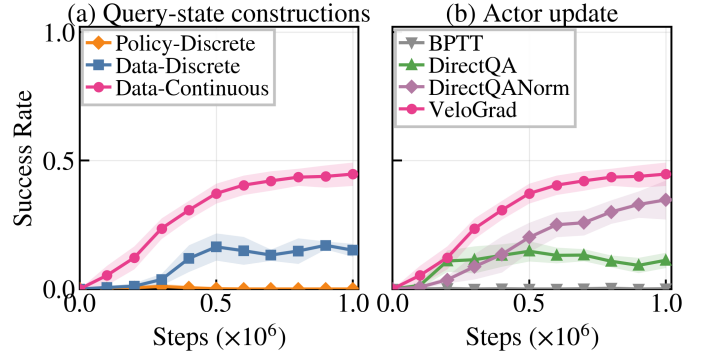


Fig. 3. Design-choice experiments on offline Cube-Triple. (a) Query-state constructions for actor updates. Data-Continuous outperforms Data-Discrete and Policy-Discrete. (b) Actor-update variants that translate terminal action value into a flow-policy update. DirectQA uses a clipped clean-action proxy, DirectQANorm applies a tuned fixed norm to its per-sample velocity-space gradient, and BPTT backpropagates through the full Euler chain; all remain below VeloGrad.

with all other training settings fixed, Data-Continuous performs substantially better on offline Cube-Triple, while Data-Discrete improves only partially and Policy-Discrete remains near zero.

To probe the role of replay anchoring, we measure twin action-critic disagreement, $|Q_1^A(s, a) - Q_2^A(s, a)|$, at terminal actions induced by the three query-state constructions as a proxy for critic uncertainty. Across the evaluated runs, Policy-Discrete consistently yields higher disagreement than Data-Discrete; replay anchoring lowers mean disagreement by approximately 8%. This is consistent with greater action-critic extrapolation exposure for current-policy continuations. Data-Continuous and Data-Discrete yield similar disagreement, so the additional performance benefit of continuous-time queries is not captured by this diagnostic.

We next compare different ways of translating terminal action value into a flow-policy update on offline Cube-Triple. DirectQA maximizes $\bar{Q}^A(s, \hat{a})$ through the clipped clean-action proxy $\hat{a} = \text{clip}(x_t^D + (1 - t)u, -1, 1)$, with $u = v_\theta(s, x_t^D, t)$. DirectQANorm rescales the resulting per-sample velocity-space gradient to a fixed norm selected by hyperparameter search. As shown in Fig. 3(b), DirectQA provides only limited improvement; the tuned fixed-norm variant improves performance but remains below VeloGrad throughout training. BPTT, which propagates terminal action value through the complete Euler generation chain, remains near zero under the same training setting. These results support the learned local velocity-value interface in this setting. We isolate the additional effect of directional sensitivity supervision in Fig. 4.

Directional sensitivity supervision. Compared with endpoint-only distillation, finite-radius sensitivity supervision lowers directional error across flow time on held-out rollouts and improves aggregate offline success over the ten Cube-Double and Cube-Triple tasks (Fig. 4(a,b)).

To probe derivative consistency at actor queries, we measure directional secant error on Data-Continuous states in Cube-Triple using each method’s own action critic and continuation.

TABLE I

OGBENCH SUCCESS RATE (%; MEAN $\pm$ STANDARD DEVIATION OVER 12 TRAINING SEEDS) AFTER 1M OFFLINE GRADIENT STEPS AND AN ADDITIONAL 1M ENVIRONMENT STEPS FOR OFFLINE-TO-ONLINE (O2O) ADAPTATION. P33: PUZZLE-3 $\times$ 3; C2: CUBE-DOUBLE; C3: CUBE-TRIPLE. RLPD IS EVALUATED ONLY IN O2O. AVG. USES THE FOUR UNROUNDED DOMAIN SCORES.

Setting	Method	Scene	P33	C2	C3	Avg.
Offline	DAC [20]	71.8 $\pm$ 1.4	72.3 $\pm$ 15.2	38.7 $\pm$ 4.6	2.1 $\pm$ 1.7	46.2 $\pm$ 3.5
	DSRL [5]	99.9 $\pm$ 0.2	85.0 $\pm$ 19.1	75.5 $\pm$ 3.7	1.6 $\pm$ 0.6	65.5 $\pm$ 5.2
	QAM [4]	97.0 $\pm$ 1.5	100.0 $\pm$ 0.0	64.3 $\pm$ 2.8	4.8 $\pm$ 1.4	66.5 $\pm$ 0.7
	QC-FQL [3], [25]	83.5 $\pm$ 0.4	54.1 $\pm$ 18.7	41.7 $\pm$ 2.3	4.2 $\pm$ 1.9	45.9 $\pm$ 5.4
	VeloGrad	98.0 $\pm$ 0.9	100.0 $\pm$ 0.0	75.8 $\pm$ 2.9	44.7 $\pm$ 4.6	79.6 $\pm$ 1.7
O2O	DAC [20]	97.7 $\pm$ 1.7	38.0 $\pm$ 9.7	99.7 $\pm$ 0.4	34.8 $\pm$ 6.0	67.5 $\pm$ 2.9
	DSRL [5]	100.0 $\pm$ 0.0	84.8 $\pm$ 19.5	99.9 $\pm$ 0.2	0.6 $\pm$ 0.8	71.3 $\pm$ 5.0
	QAM [4]	99.6 $\pm$ 0.6	100.0 $\pm$ 0.0	99.9 $\pm$ 0.2	79.4 $\pm$ 0.7	94.7 $\pm$ 0.1
	QC-FQL [3], [25]	99.0 $\pm$ 1.0	100.0 $\pm$ 0.0	99.7 $\pm$ 0.6	58.1 $\pm$ 2.9	89.2 $\pm$ 0.9
	RLPD [9]	92.8 $\pm$ 1.9	100.0 $\pm$ 0.0	98.7 $\pm$ 1.1	40.0 $\pm$ 7.4	82.9 $\pm$ 1.8
	VeloGrad	98.0 $\pm$ 0.9	100.0 $\pm$ 0.0	99.6 $\pm$ 0.4	93.4 $\pm$ 7.1	97.7 $\pm$ 1.8

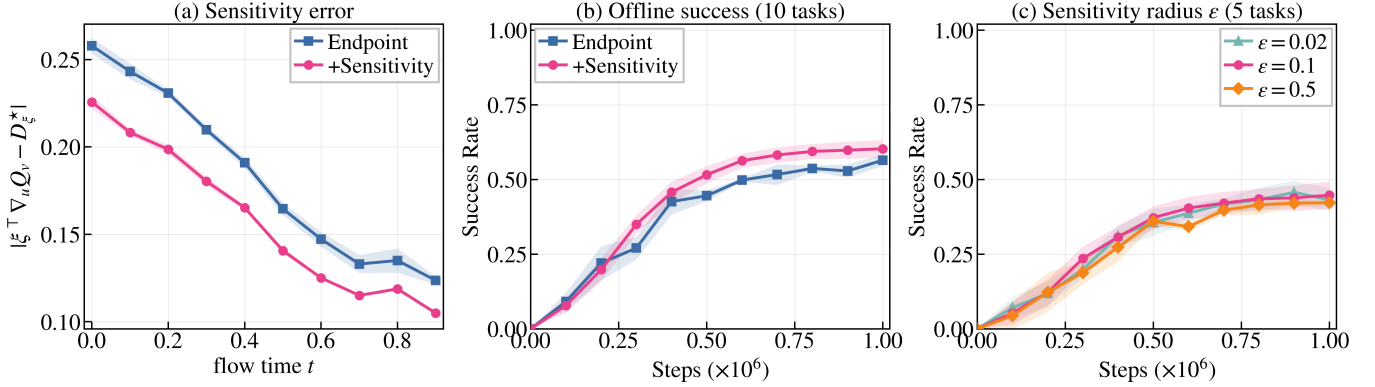


Fig. 4. Effect of finite-radius sensitivity supervision. Endpoint uses endpoint-only value distillation; +Sensitivity additionally includes $\mathcal{L}_{\text{sens}}$. (a) Directional error $|\xi^\top \nabla_u Q_u - D_u^*|$ on held-out rollouts versus flow time. (b) Aggregate offline success over the ten Cube-Double and Cube-Triple tasks. (c) Mean success over the five Cube-Triple tasks for different perturbation radii ϵ .

TABLE II

MEAN TIME PER TRAINING STEP OVER 10,000 STEPS ON OGBENCH CUBE-TRIPLE TASK 1 USING A SINGLE NVIDIA RTX 4090D GPU, BATCH SIZE 256, AND ACTION-CHUNK HORIZON $H = 5$ FOR ALL METHODS.

Method	Time (ms/step)
QC-FQL	5.228
QAM	7.034
VeloGrad w/o sensitivity	6.065
VeloGrad	7.048

Sensitivity supervision reduces mean absolute error by approximately 16% relative to endpoint-only distillation, indicating that the observed error reduction extends beyond policy-rollout states to actor queries.

We further evaluate perturbation radii $\epsilon \in \{0.02, 0.1, 0.5\}$ on the five Cube-Triple tasks (Fig. 4(c)). The two smaller radii achieve similar success rates, while $\epsilon = 0.5$ yields a lower final mean, supporting $\epsilon = 0.1$ as a reasonable default.

Training cost. VeloGrad has comparable per-step training cost to QAM; sensitivity supervision adds 16.2% overhead over endpoint-only training (Table II).

B. Real-Robot Offline-to-Online RL

Setup and protocol. We evaluate VeloGrad for real-world offline-to-online reinforcement learning on a Piper-X robot. The policy observes RGB images from a wrist-mounted camera and a fixed third-person camera, together with the end-effector pose and gripper state. Images are captured at 720×480 , cropped using task-specific fixed regions, resized to 128×128 , and encoded by a ResNet-10 visual backbone.

All tasks use an action-chunk flow policy that predicts 6-DoF Cartesian ΔEEF motion and gripper commands, with $H = 16$ for Flower Insertion and Elastic Puck and $H = 32$ for Windmill Activation. The environment runs at 25 Hz. We use synchronous chunk-wise execution: the policy replans from a new observation only after executing the complete H -step action chunk. Cartesian workspace bounds and per-step delta-action bounds are enforced throughout data collection, online training, and evaluation.

For each real-robot task, we collect 10 teleoperated demonstrations and pretrain the action-chunk flow policy for 30,000 gradient steps using flow matching. The same demonstrations initialize the replay buffer. USB insertion in Windmill Activation is demonstrated using an Xbox controller, while the remaining behaviors use a leader arm. During online

adaptation, the current policy controls the robot by default. When correction is required, the operator takes over through the teleoperation interface; the corrective action replaces the policy action, and the resulting transition enters the replay buffer for subsequent off-policy updates. All online adaptation methods use this human-correction protocol during training.

Each online adaptation method receives a budget of 100 online trajectories per task. Checkpoints are saved every 25 online trajectories and evaluated after training. The scene is manually reset between trials.

Network architecture. For VeloGrad, the actor and critics use separate per-camera ResNet-10 encoders [36] with spatial-softmax heads [37]. Both sets of visual backbones are initialized from the same ImageNet-pretrained ResNet-10 checkpoint, while keeping their parameters separate. Each camera is encoded into a 64-dimensional feature; the two visual features are concatenated with the end-effector pose and gripper state. The actor uses a conditional 1D U-Net with channel widths [128, 256, 512]. Q^A and Q_v share the critic-side encoder, but each has two independent MLP heads with three 512-unit hidden layers. The actor backbones are trained at $0.1 \times$ the actor learning rate; the critic backbones are frozen, while their spatial-softmax heads remain trainable.

Online training. Real-robot adaptation uses a continuously running actor–learner architecture. The replay buffer is initialized with the offline demonstrations and subsequently augmented with online transitions; minibatches are sampled directly from the shared replay buffer without distinguishing offline and online data. For VeloGrad, the base actor and critic learning rates are 3×10^{-4} , and minibatches contain 256 samples in both offline and online training. Each online learner iteration comprises four critic-only updates followed by one joint actor–critic update, giving five updates of each critic per actor update. Each critic update jointly optimizes Q^A and Q_v . The learner completes approximately one outer training iteration per environment step for Flower Insertion and Elastic Puck, and three for Windmill Activation. The learner publishes the latest parameters to the actor every 200 critic updates.

Training uses the reward

$$r_t = \begin{cases} 10, & \text{full-task success,} \\ 2, & \text{first Windmill } S_1/S_2 \text{ completion,} \\ 0, & \text{otherwise,} \end{cases} \quad (13)$$

with discount factor $\gamma = 0.98$. In Windmill Activation, completing USB insertion or sand-tray positioning yields +2 once per stage per episode. Completing the final windmill stage yields only the full-task success reward of +10, without an additional stage bonus. All other steps, including failure and timeout transitions, receive zero reward. Task success and failure are annotated through operator button presses, using the same physical success criteria for training and evaluation.

Tasks. Fig. 5 illustrates Flower Insertion, Elastic Puck Launching, and Windmill Activation.

Flower Insertion. The robot starts each episode holding a flower, while the vase is initialized within a $10 \text{ cm} \times 10 \text{ cm}$

TABLE III
FINAL REAL-ROBOT AUTONOMOUS SUCCESSES OVER 30 EVALUATION TRIALS PER METHOD AND CONDITION. SUCCESS REQUIRES TASK COMPLETION WITHOUT HUMAN INTERVENTION; BOLD DENOTES THE BEST RESULT PER CONDITION.

Method	Flower		Windmill	
	Static	Disturbed	Puck	E2E
BC Init	8/30	3/30	0/30	0/30
HG-DAGger [38]	21/30	23/30	9/30	7/30
DSRL [5]	24/30	7/30	8/30	16/30
QAM [4]	26/30	23/30	10/30	15/30
VeloGrad	29/30	29/30	13/30	21/30

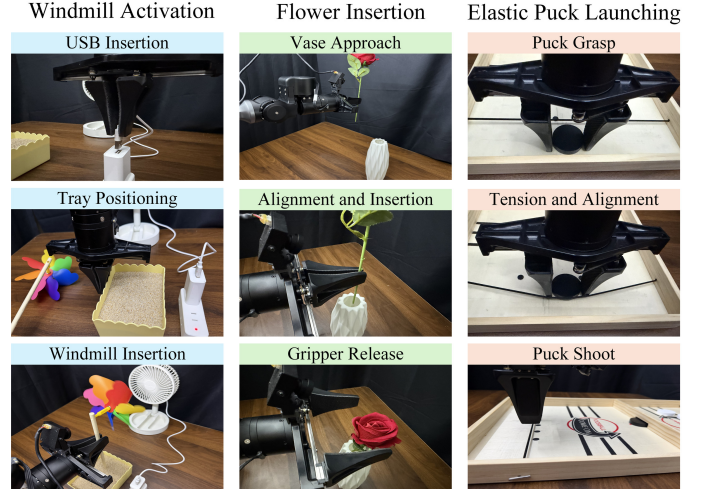


Fig. 5. Piper-X tasks: Windmill Activation, Flower Insertion, and Elastic Puck Launching (columns), each shown at three representative stages.

region on the table. The policy must visually align the flower stem with the vase opening and complete the insertion. Training includes vase displacements. In **Flower-Static** evaluation, the vase remains stationary throughout execution. In **Flower-Disturbed** evaluation, the vase is externally displaced once per trial, as the robot begins moving, to a randomly selected position within the reachable workspace, requiring re-localization and a corrected approach. Each episode is limited to 400 environment steps.

Elastic Puck Launching. A single puck is initialized at a random position within a designated pickup region. Success requires grasping the puck and using an elastic band to launch it through the central opening into the opposite region within 500 environment steps. Evaluation counts and initial-state distributions are matched across methods.

Windmill Activation. The robot starts holding a USB plug and inserts it into a fixed socket to activate the fan (S_1). It then pushes a sand tray, initialized within a $5 \text{ cm} \times 5 \text{ cm}$ region, directly in front of the fan (S_2). Finally, it grasps a windmill, inserts it into the sand tray, and positions it to rotate under the airflow (S_3). The windmill starts within another $5 \text{ cm} \times 5 \text{ cm}$ region, with planar orientation sampled from $[-20^\circ, 20^\circ]$.

Autonomous E2E success requires completing the entire sequence within 1000 environment steps without human in-

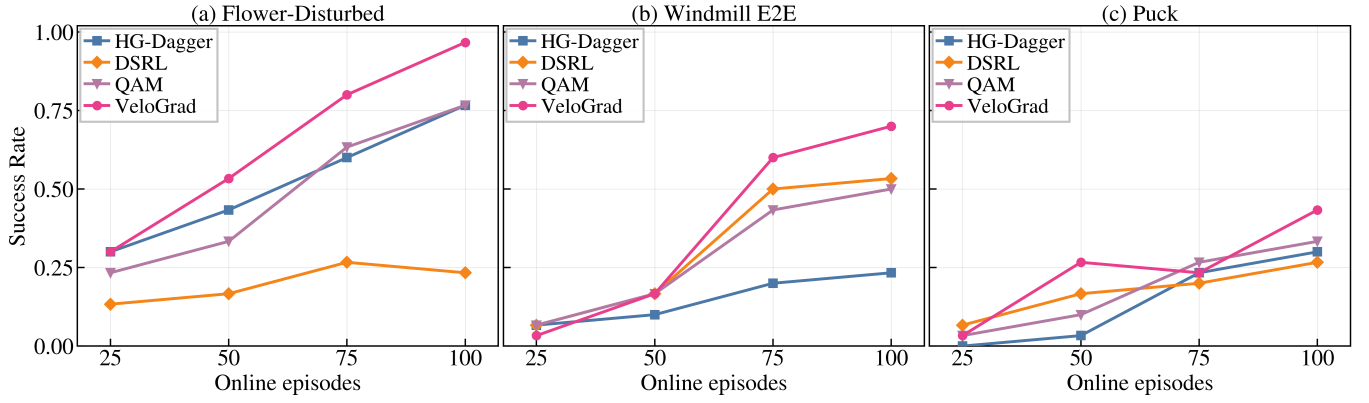


Fig. 6. Autonomous success on (a) Flower-Disturbed, (b) Windmill E2E, and (c) Puck. HG-Dagger, DSRL, QAM, and VeloGrad checkpoints from 25, 50, 75, and 100 online training episodes are evaluated after training, with 30 trials per condition. Success requires task completion without intervention.

tervention.

Evaluation protocol. Table III reports 30 autonomous trials per condition for a single final policy checkpoint of each method. Fig. 6 evaluates the 25-, 50-, 75-, and 100-trajectory checkpoints after training, each for 30 trials per condition. Autonomous success requires completing the entire task without corrective teleoperation or manual recovery.

Windmill stage-wise diagnostics use the same 30 final-checkpoint evaluation trials. Each stage is attempted autonomously. If a stage fails, it is recorded as unsuccessful, and the operator performs recovery to enable evaluation of the next stage. A stage’s success rate is the fraction of all 30 trials in which that stage is completed without correction, regardless of earlier stage outcomes. Any trial requiring recovery remains an E2E failure.

Comparisons. We compare VeloGrad against **BC Init**, the 30,000-step flow-matching policy trained on 10 task demonstrations; **HG-Dagger**, a human-corrective online imitation-learning baseline [38]; and DSRL and QAM, alternative reinforcement-learning interfaces for pretrained policies [4], [5]. All online methods share the demonstrations, task definitions, rewards, trajectory budget, action frequency, chunk horizon, training correction protocol, and evaluation protocol, including stage-wise recovery. They retain method-specific optimization and replay strategies.

Results. Final autonomous performance. VeloGrad achieves the highest success count in all four evaluation conditions (Table III). The largest gain over the strongest baseline is on Flower-Disturbed: 29/30 versus 23/30 for QAM and HG-Dagger. Flower insertion also reaches 29/30 with static targets. On Windmill Activation, VeloGrad completes 21/30 autonomous successes versus 16/30 for DSRL. On puck launching, VeloGrad completes 13/30 trials versus 10/30 for QAM.

Online adaptation. VeloGrad exceeds HG-Dagger and QAM on Flower-Disturbed at the 50-, 75-, and 100-episode checkpoints (Fig. 6). On Windmill E2E, VeloGrad and QAM perform similarly at 50 episodes, with VeloGrad ahead at 75 and 100 episodes. VeloGrad continues to improve through the

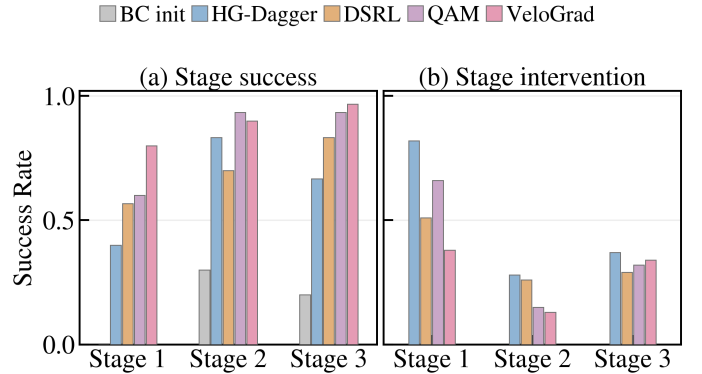


Fig. 7. Windmill diagnostics: S_1 , USB insertion; S_2 , sand-tray positioning; S_3 , windmill grasping and insertion until rotation. (a) Autonomous stage success over the same 30 final-checkpoint trials. Human recovery after a failed stage enables the next stage’s evaluation; the failed stage and E2E remain unsuccessful. (b) Human-correction rates over 100 online training trajectories, counting a trajectory as intervened at a stage if the operator took over there. BC Init appears only in (a), as it has no online adaptation.

final checkpoint on both tasks.

Stage-wise performance. Fig. 7(a) shows that VeloGrad achieves the highest success in USB insertion and in the final windmill grasp-and-insert stage, whose success requires rotation. QAM achieves the highest success in sand-tray positioning. The largest stage-wise difference between VeloGrad and QAM occurs in USB insertion. Fig. 7(b) reports the corresponding human-correction rates over the 100 online training trajectories. VeloGrad required corrections on 38/100 trajectories at USB insertion, compared with 82/100 for HG-Dagger, 66/100 for QAM, and 51/100 for DSRL. Stage 2 and Stage 3 intervention rates for VeloGrad remain comparable or lower (13/100 and 34/100).

V. CONCLUSION

VeloGrad learns the continuation value of a local flow velocity and supervises the derivatives used for actor improvement with symmetric finite-radius perturbations. This enables local actor updates without backpropagating through the remaining solver and preserves the standard deployment sampler.

Simulation and real-robot experiments demonstrate improved performance, including 21/30 autonomous successes on long-horizon Windmill Activation.

REFERENCES

- [1] C. Chi *et al.*, “Diffusion Policy: Visuomotor Policy Learning via Action Diffusion,” in *Proc. Robot.: Sci. Syst. (RSS)*, 2023.
- [2] Y. Lipman, R. T. Q. Chen, H. Ben-Hamu, M. Nickel, and M. Le, “Flow Matching for Generative Modeling,” in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2023.
- [3] S. Park, Q. Li, and S. Levine, “Flow Q-Learning,” in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2025, pp. 48104–48127.
- [4] Q. Li and S. Levine, “Q-Learning with Adjoint Matching,” in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2026.
- [5] A. Wagenmaker *et al.*, “Steering Your Diffusion Policy with Latent Space Reinforcement Learning,” in *Proc. Conf. Robot Learn. (CoRL)*, 2025, pp. 258–282.
- [6] R. Yang *et al.*, “VINE: Taming Generative Control Policies for Reinforcement Learning,” *arXiv preprint arXiv:2607.10369*, 2026.
- [7] K. Shi *et al.*, “FlowDPG: Deterministic Policy Gradient on Flow Matching Policies for Real-World Manipulation,” *arXiv preprint arXiv:2606.22303*, 2026.
- [8] Z. Wang, Y. Liu, X. Mao, M. Wang, and Y. Mu, “Q-VGM: Q-Value-Gradient Matching for Offline-to-Online Reinforcement Learning of Flow-Matching VLA,” *arXiv preprint arXiv:2606.08015*, 2026.
- [9] P. J. Ball, L. Smith, I. Kostrikov, and S. Levine, “Efficient Online Reinforcement Learning with Offline Data,” in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2023, pp. 1577–1594.
- [10] J. Luo *et al.*, “SERL: A Software Suite for Sample-Efficient Robotic Reinforcement Learning,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2024.
- [11] J. Luo, C. Xu, J. Wu, and S. Levine, “Precise and Dexterous Robotic Manipulation via Human-in-the-Loop Reinforcement Learning,” *Science Robotics*, vol. 10, no. 105, Art. no. eads5033, 2025.
- [12] L. Ankile, Z. Jiang, R. Duan, G. Shi, P. Abbeel, and A. Nagabandi, “Residual Off-Policy RL for Finetuning Behavior Cloning Policies,” *arXiv preprint arXiv:2509.19301*, 2025.
- [13] K. Lei *et al.*, “Performant robotic manipulation with real-world reinforcement learning,” *Science Robotics*, vol. 11, no. 116, Art. no. ead6267, 2026.
- [14] Y. Chen, S. Tian, S. Liu, Y. Zhou, H. Li, and D. Zhao, “ConRFT: A Reinforced Fine-tuning Method for VLA Models via Consistency Policy,” in *Proc. Robot.: Sci. Syst. (RSS)*, 2025.
- [15] P. Dong, K.-H. Hung, T. Gao, D. Sadigh, and C. Finn, “EXPO-FT: Sample-Efficient Reinforcement Learning Finetuning for Vision-Language-Action Models,” *arXiv preprint arXiv:2605.25477*, 2026.
- [16] C. Xu *et al.*, “RL Token: Bootstrapping Online RL with Vision-Language-Action Models,” *arXiv preprint arXiv:2604.23073*, 2026.
- [17] Physical Intelligence *et al.*, “ π^*_0 : a VLA That Learns From Experience,” *arXiv preprint arXiv:2511.14759*, 2025.
- [18] W. Xiao *et al.*, “Self-Improving Vision-Language-Action Models with Data Generation via Residual RL,” in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2026.
- [19] Z. Wang, J. J. Hunt, and M. Zhou, “Diffusion Policies as an Expressive Policy Class for Offline Reinforcement Learning,” in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2023.
- [20] L. Fang, R. Liu, J. Zhang, W. Wang, and B.-Y. Jing, “Diffusion Actor-Critic: Formulating Constrained Policy Iteration as Diffusion Noise Regression for Offline Reinforcement Learning,” in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2025.
- [21] A. Z. Ren *et al.*, “Diffusion Policy Policy Optimization,” in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2025.
- [22] D. McAllister *et al.*, “Flow Matching Policy Gradients,” in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2026.
- [23] T. Zhang, C. Yu, S. Su, and Y. Wang, “ReinFlow: Fine-tuning Flow Matching Policy with Online Reinforcement Learning,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- [24] F. Nguimatsia Tiofack, T. Le Hellard, F. Schramm, N. Perrin-Gilbert, and J. Carpentier, “Guided Flow Policy: Learning from High-Value Actions in Offline Reinforcement Learning,” in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2026.
- [25] Q. Li, Z. Zhou, and S. Levine, “Reinforcement Learning with Action Chunking,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2025.
- [26] Z. Sun and S. Song, “From Prior to Pro: Efficient Skill Mastery via Distribution Contractive RL Finetuning,” *arXiv preprint arXiv:2603.10263*, 2026.
- [27] P. Dong, Q. Li, D. Sadigh, and C. Finn, “EXPO: Stable Reinforcement Learning with Expressive Policies,” in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2026.
- [28] D. Yu, K. Lei, Z. Jiang, J. Pan, and H. Xu, “Beyond Action Residuals: Real-World Robot Policy Steering via Bottleneck Latent Reinforcement Learning,” *arXiv preprint arXiv:2605.19919*, 2026.
- [29] D. Silver, G. Lever, N. Heess, T. Degris, D. Wierstra, and M. Riedmiller, “Deterministic Policy Gradient Algorithms,” in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2014, pp. 387–395.
- [30] M. Psenka, A. Escontrela, P. Abbeel, and Y. Ma, “Learning a Diffusion Model Policy from Rewards via Q-Score Matching,” in *Proc. Int. Conf. Mach. Learn. (ICML)*, 2024, pp. 41163–41182.
- [31] H. Im, A. Kolobov, J. Fu, and Y. Lee, “Latent Policy Steering through One-Step Flow Policies,” in *Proc. Robot.: Sci. Syst. (RSS)*, 2026.
- [32] W. M. Czarnecki, S. Osindero, M. Jaderberg, G. Swirszcz, and R. Pascanu, “Sobolev Training for Neural Networks,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2017.
- [33] A. D. Flaxman, A. T. Kalai, and H. B. McMahan, “Online Convex Optimization in the Bandit Setting: Gradient Descent Without a Gradient,” in *Proc. ACM-SIAM Symp. Discrete Algorithms (SODA)*, 2005, pp. 385–394.
- [34] S. Park, K. Frans, B. Eysenbach, and S. Levine, “OGBench: Benchmarking Offline Goal-Conditioned RL,” in *Proc. Int. Conf. Learn. Representations (ICLR)*, 2025.
- [35] A. Mandlekar *et al.*, “What Matters in Learning from Offline Human Demonstrations for Robot Manipulation,” in *Proc. Conf. Robot Learn. (CoRL)*, 2022, pp. 1678–1690.
- [36] K. He, X. Zhang, S. Ren, and J. Sun, “Deep Residual Learning for Image Recognition,” in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit. (CVPR)*, 2016, pp. 770–778.
- [37] S. Levine, C. Finn, T. Darrell, and P. Abbeel, “End-to-End Training of Deep Visuomotor Policies,” *J. Mach. Learn. Res.*, vol. 17, no. 39, pp. 1–40, 2016.
- [38] M. Kelly, K. Sidrane, K. Driggs-Campbell, and M. J. Kochenderfer, “HG-DAGger: Interactive Imitation Learning with Human Experts,” in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, 2019, pp. 8077–8083.